

AI PDF Assistant & Document Intelligence

SUBMITTED BY	ENVIRONMENT	HOST PORT	VERIFICATION
Mohammed Kaderji	Jupyter • Streamlit • PyPDF	localhost:8503	✓ Verified & Tested

1. Objective & Folder Structure

Objective: Build an interactive document reader that parses PDF files, answers questions strictly grounded in the document text, and performs one-click summarization and multilingual translation using the Gemini API.

Folder Structure: AI_PDF_Assistant/ → app.py, requirements.txt, README.md

2. Step-by-Step Implementation Guide

Step 1: Install Required Libraries

In Jupyter Notebook, install Streamlit, Google Generative AI, and PyPDF:

```
!pip install streamlit google-generativeai pypdf
```

Press **Shift + Enter**. The pypdf library extracts raw text from PDF files.

Step 2: Create Folder & Files

Inside Jupyter, create the dedicated folder and requirements file:

- Run `import os; os.makedirs("AI_PDF_Assistant", exist_ok=True)`
- Run `%%writefile AI_PDF_Assistant/requirements.txt` containing streamlit, google-generativeai, and pypdf.

Step 3: Write app.py (Document Pipeline + Challenge Buttons)

Write the application using `%%writefile AI_PDF_Assistant/app.py`.

Solving the Project Challenge: Integrated PdfReader loop, in-context Q&A, and built two quick-action buttons: **Summarize PDF** and **Translate PDF** (Hindi, French, German, Spanish).

Step 4: Run Application (Server Launch)

Open Command Prompt (cmd), navigate to the directory, and start Streamlit on port 8503 to avoid conflicts:

```
cd C:/Users/Hp/AI_PDF_Assistant
```

```
streamlit run app.py --server.port 8503
```

Step 5: Verify Live Output

Open browser at `http://localhost:8503`, upload `ai_tools.pdf`, select **Hindi**, and click **Translate PDF** to verify real-time translation output.

3. Source Code Implementation (`app.py`)

```
import streamlit as st
from pypdf import PdfReader
import google.generativeai as genai

# 1. Configure Gemini 3.6 Flash Credentials
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-3.6-flash")

# 2. Application UI Title & File Uploader
st.title("📄 AI PDF Assistant")
st.write("Upload a PDF document to analyze, summarize, or ask specific questions.")

uploaded_file = st.file_uploader("Upload PDF", type="pdf")

if uploaded_file:
    # Extract text from all pages
    pdf = PdfReader(uploaded_file)
    extracted_text = ""
    for page in pdf.pages:
        extracted_text += page.extract_text() or ""
    st.success(f"File uploaded successfully! Total pages: {len(pdf.pages)}")

    # Section A: Document Q&A with Strict In-Context Grounding
    question = st.text_input("Ask a Question about the document")
    if st.button("Answer Question"):
        if question:
            prompt = f"""Read the document. Answer only using this document.
Document:
{extracted_text}
Question:
{question}"""
            with st.spinner("Analyzing..."):
                response = model.generate_content(prompt)
                st.markdown("### Answer:")
                st.write(response.text)
        else:
            st.warning("Please type a question.")

    st.divider()

    # Section B: Challenge Buttons (Summarize & Multilingual Translation)
    st.subheader("Quick Actions")
    col1, col2 = st.columns(2)

    with col1:
        if st.button("📄 Summarize PDF"):
            summary_prompt = f"""Summarize key points of this document concisely in bullet points:
{extracted_text}"""
            with st.spinner("Summarizing..."):
                res = model.generate_content(summary_prompt)
                st.markdown("### Summary:")
                st.write(res.text)

    with col2:
        target_lang = st.selectbox("Translate to", ["Hindi", "French", "German", "Spanish"])
        if st.button("🌐 Translate PDF"):
            trans_prompt = f"""Translate the core contents and main summary of this document into
{target_lang}:
{extracted_text}"""
            with st.spinner(f"Translating to {target_lang}..."):
                res = model.generate_content(trans_prompt)
                st.markdown(f"### Translated ({target_lang}):")
                st.write(res.text)
```

4. Live Execution Proof & Verification (Port 8503)

Evaluation & Test Parameters:

- **Target Host:** `http://localhost:8503` (Dedicated active local server instance).
- **Uploaded File:** `ai_tools.pdf` (2 Pages ingested and extracted successfully).
- **Challenge Execution:** Executed **Translate PDF** action targeted to **Hindi**.
- **Observed Output:** Real-time extraction and synthesis into fluent Hindi covering मुख्य सारांश (*Main Summary*) and मुख्य सामग्री (*Core Contents*) for NotebookLM, Gemini, Google AI Studio, Stitch, Pomelli, and Opal.

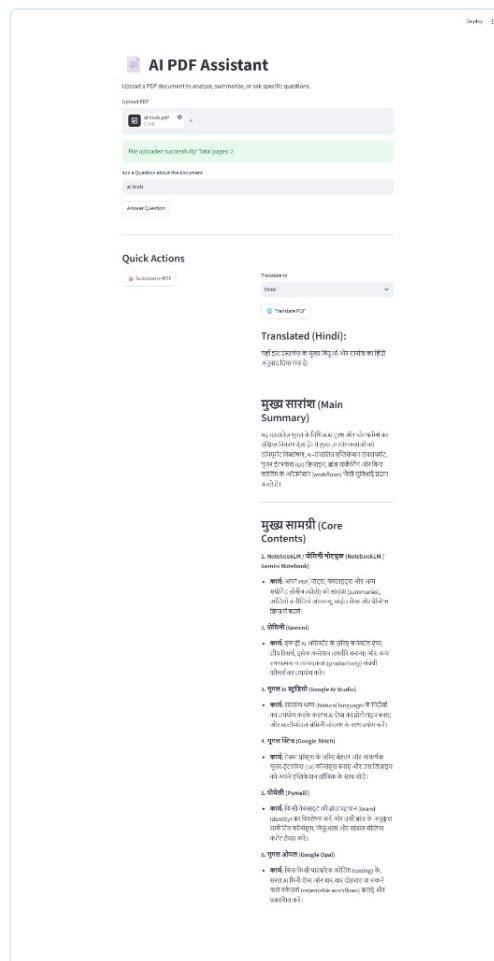


Figure 1.0: Live web browser proof verifying PDF ingestion, UI controls, and real-time Hindi translation output.